

নিজের Dataset তৈরি করা

অনেক সময় তুমি যে **NLP (Natural Language Processing)** অ্যাপ্লিকেশন বানাতে চাও, সেই ধরনের **dataset** আগে থেকে তৈরি থাকে না। তখন তোমাকে নিজেই dataset তৈরি করতে হয়।

এই অংশে দেখানো হচ্ছে কীভাবে **GitHub issues** থেকে একটি **corpus** (ডেটার সংগ্রহ) তৈরি করা যায়।

GitHub-এ **issue** সাধারণত ব্যবহার করা হয়:

- bug রিপোর্ট করার জন্য
- নতুন feature প্রস্তাব করার জন্য
- প্রশ্ন করার জন্য

এই ধরনের issue ডেটা ব্যবহার করে বিভিন্ন কাজ করা যায়।

এই **dataset** দিয়ে কী করা যায়

📌 **Issue** বা **Pull Request** বন্ধ হতে কত সময় লাগে তা বিশ্লেষণ করা

📌 **Multilabel classifier** ট্রেন করা

যাতে issue এর description দেখে স্বয়ংক্রিয়ভাবে label দিতে পারে যেমন:

- bug
- enhancement
- question

📌 **Semantic Search Engine** তৈরি করা

যেখানে user কোনো প্রশ্ন লিখলে সেই অনুযায়ী সম্পর্কিত GitHub issue খুঁজে পাওয়া যাবে।

এই অংশে আমরা মূলত **dataset** তৈরি করা শিখব।

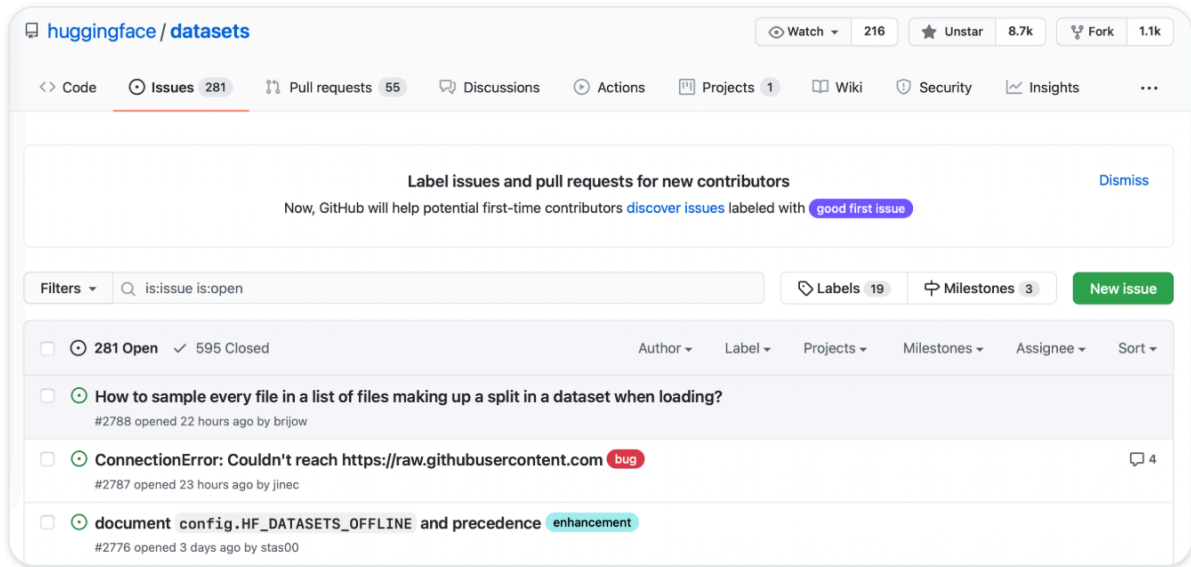
পরবর্তী অংশে দেখানো হবে **semantic search application**।

এখানে উদাহরণ হিসেবে ব্যবহার করা হয়েছে একটি জনপ্রিয় open-source project:

HuggingFace Datasets

5.5.1 Data সংগ্রহ করা (Getting the Data)

GitHub repository তে গেলে **Issues tab** এ সব issue দেখা যায়।



যেমন উদাহরণ হিসেবে:

- **331 টি open issue**
- **668 টি closed issue**

প্রতিটি issue এর মধ্যে থাকে:

- title
- description
- labels
- creator
- comments
- Status



GitHub API ব্যবহার করে Issues ডাউনলোড করা

সব issue ডাউনলোড করার জন্য আমরা ব্যবহার করব:

GitHub REST API

এই API একটি **JSON object list** রিটার্ন করে যেখানে প্রতিটি object একটি issue।

এগুলোর মধ্যে থাকে:

- title
- description
- status
- user information
- metadata

Python দিয়ে GitHub API ব্যবহার করা

আমরা Python এর **requests library** ব্যবহার করব HTTP request পাঠানোর জন্য।

Library install

!pip install requests

API Request পাঠানো

```
import requests
```

```
url = "https://api.github.com/repos/huggingface/datasets/issues?page=1&per_page=1"  
response = requests.get(url)
```

এই request দিয়ে আমরা প্রথম পেজের প্রথম issue পাব।

HTTP Status Code

```
response.status_code
```

Output:

200

২০০ মানে **request** সফল হয়েছে।

JSON Data দেখা

```
response.json()
```

এখানে একটি বড় JSON object পাওয়া যায়।

এর মধ্যে কিছু গুরুত্বপূর্ণ field হলো:

Field	মানে
title	issue এর শিরোনাম
body	issue এর description
number	issue নম্বর

user কে issue খুলেছে

state open বা closed

created_at issue কখন তৈরি হয়েছে

comments কয়টা comment আছে

GitHub API Rate Limit

GitHub API ব্যবহার করার সময় একটি সীমা আছে।

Token ছাড়া

প্রতি ঘন্টায় মাত্র

60 request

করা যায়।

Token ব্যবহার করলে

5000 request প্রতি ঘন্টায়

করতে পারো।

Personal Access Token ব্যবহার করা

GitHub token তৈরি করার পর request header এ যোগ করতে হবে।

```
GITHUB_TOKEN = xxx
```

```
headers = {"Authorization": f"token {GITHUB_TOKEN}"}
```

সতর্কতা

তোমার **GitHub token** কখনো **public notebook** এ শেয়ার করা উচিত না।

ভালো উপায় হলো:

- `.env file` এ রাখা
- `python-dotenv` দিয়ে load করা

সব Issue Download করার Function

এরপর একটি Python function বানানো হয়েছে:

```
fetch_issues()
```

এই function:

- GitHub API call করবে
- issue গুলো batch আকারে download করবে
- rate limit এ পৌঁছালে ১ ঘন্টা অপেক্ষা করবে
- শেষে সব data save করবে

ফাইল আকারে:

```
datasets-issues.jsonl
```

Function Run করা

```
fetch_issues()
```

ইন্টারনেট স্পিড অনুযায়ী কয়েক মিনিট লাগতে পারে।

Dataset Load করা

```
issues_dataset = load_dataset(  
    "json",  
    data_files="datasets-issues.jsonl",  
    split="train"  
)
```

Output:

```
Dataset({  
  features: [...],  
  num_rows: 3019  
})
```

মানে dataset এ আছে **3019 rows**।

কেন **Issue** সংখ্যা বেশি?

GitHub issue tab এ হয়তো প্রায় **1000 issue** দেখায়।

কিন্তু dataset এ এসেছে **3000+**।

কারণ:

GitHub API তে **Pull Request** কে **Issue** হিসেবে গণনা করা হয়।

অর্থাৎ:

- সব Pull Request = Issue
- কিন্তু সব Issue = Pull Request না

Pull Request চেনার উপায়

JSON data তে যদি থাকে:

pull_request

তাহলে বুঝতে হবে এটি **Pull Request**।

5.5.2 Cleaning up the Data

GitHub documentation অনুযায়ী আমরা জানি যে **pull_request column** ব্যবহার করে বোঝা যায় একটি entry:

- **Issue**
নাকি
- **Pull Request**

অর্থাৎ dataset-এ issue এবং pull request দুটোই এসেছে, তাই এগুলো আলাদা করতে হবে।

Random Sample দেখে পার্থক্য বোঝা

প্রথমে dataset থেকে কিছু **random data** নিয়ে দেখা হচ্ছে।

```
sample = issues_dataset.shuffle(seed=666).select(range(3))
```

এখানে দুইটি কাজ হচ্ছে:

1. **shuffle()**

Dataset কে random ভাবে মিশিয়ে দেয়।

2. **select(range(3))**

শুধু প্রথম ৩টা **sample** নেয়।

এরপর URL এবং pull_request column একসাথে দেখা হচ্ছে:

```
for url, pr in zip(sample["html_url"], sample["pull_request"]):  
    print(f">> URL: {url}")  
    print(f">> Pull request: {pr}\n")
```

এখানে:

zip() ব্যবহার করা হয়েছে দুইটা column একসাথে iterate করার জন্য।

Output ব্যাখ্যা

উদাহরণ:

URL: <https://github.com/huggingface/datasets/pull/850>
Pull request: { ... }

এখানে দেখা যাচ্ছে:

- URL এ **pull** লেখা আছে
- pull_request field এ **data** আছে

অর্থাৎ এটা **Pull Request**।

আরেকটা উদাহরণ:

URL: <https://github.com/huggingface/datasets/issues/2773>
Pull request: None

এখানে:

- URL এ **issues**
- pull_request = **None**

অর্থাৎ এটা সাধারণ **Issue**।

মূল পার্থক্য

pull_request field	অর্থ
None	Issue
data আছে	Pull Request

নতুন Column তৈরি করা

এখন আমরা dataset এ একটি নতুন column বানাবো:

is_pull_request

```
issues_dataset = issues_dataset.map(  
    lambda x: {"is_pull_request": False if x["pull_request"] is None else True}  
)
```

এটা কী করছে?

এই code প্রতিটি row চেক করছে।

যদি:

```
pull_request == None
```

তাহলে

```
is_pull_request = False
```

অর্থাৎ এটা **issue**।

আর যদি None না হয়:

```
is_pull_request = True
```

অর্থাৎ এটা **pull request**।

5.5.3 Cleaning up the Data

GitHub-এর ডকুমেন্টেশনের আগের অংশ থেকে আমরা জানতে পারি যে **pull_request** কলাম ব্যবহার করে বোঝা যায় একটি রেকর্ড সাধারণ **issue** নাকি **pull request**। এখন আমরা কয়েকটি **random sample** দেখে এই পার্থক্যটা বুঝব।

এখানে আমরা **Dataset.shuffle()** এবং **Dataset.select()** ব্যবহার করে dataset থেকে কিছু এলোমেলো নমুনা নেব। তারপর **html_url** এবং **pull_request** কলাম একসাথে দেখব যাতে URL এবং pull request-এর তথ্য তুলনা করা যায়।

Random sample তৈরি করা

```
sample = issues_dataset.shuffle(seed=666).select(range(3))
```

```
# URL এবং pull request তথ্য দেখানো
for url, pr in zip(sample["html_url"], sample["pull_request"]):
    print(f">> URL: {url}")
    print(f">> Pull request: {pr}\n")
```

উদাহরণ আউটপুট

```
>> URL: https://github.com/huggingface/datasets/pull/850
>> Pull request: {...}

>> URL: https://github.com/huggingface/datasets/issues/2773
>> Pull request: None

>> URL: https://github.com/huggingface/datasets/pull/783
>> Pull request: {...}
```

এখানে কী দেখা যাচ্ছে

এই আউটপুট থেকে গুরুত্বপূর্ণ দুটি বিষয় বোঝা যায়:

1. **Pull Request** হলে
 - URL-এ সাধারণত **/pull/** থাকে
 - **pull_request** ফিল্ডে একটি **object** (তথ্য) থাকে
2. সাধারণ **Issue** হলে
 - URL-এ **/issues/** থাকে
 - **pull_request** ফিল্ডের মান **None** থাকে

অর্থাৎ **pull_request** ফিল্ড ব্যবহার করে খুব সহজে বোঝা যায় কোনটি issue আর কোনটি pull request।

নতুন কলাম তৈরি করা

এই পার্থক্য ব্যবহার করে আমরা dataset-এ একটি নতুন কলাম যোগ করতে পারি: **is_pull_request**।

```
issues_dataset = issues_dataset.map(  
    lambda x: {"is_pull_request": False if x["pull_request"] is None else True}  
)
```

এই কোড কী করছে

প্রতিটি row পরীক্ষা করে:

- যদি **pull_request** **None** হয় → **is_pull_request = False**
(মানে এটি একটি **Issue**)
- যদি **pull_request** **None** না হয় → **is_pull_request = True**
(মানে এটি একটি **Pull Request**)

এভাবে dataset-এ issue এবং pull request সহজে আলাদা করা যায়।

একটি অনুশীলন (Try it out)

এখন একটি ছোট বিশ্লেষণ করার চেষ্টা করতে বলা হয়েছে।

কাজ:

Datasets রিপোজিটরিতে **issue** বন্ধ হতে গড়ে কত সময় লাগে তা বের করা।

এখানে কয়েকটি ফাংশন কাজে লাগতে পারে:

Dataset.filter()

এটি ব্যবহার করে:

- Pull request বাদ দেওয়া
- এখনও open থাকা issue বাদ দেওয়া

Dataset.set_format()

এটি ব্যবহার করে dataset কে **DataFrame** এ রূপান্তর করা যায়।

এরপর **created_at** এবং **closed_at** সময় ব্যবহার করে issue বন্ধ হতে কত সময় লাগে তা হিসাব করা যায়।

Bonus Task

Pull request বন্ধ হতে গড়ে কত সময় লাগে সেটাও বের করার চেষ্টা করতে বলা হয়েছে।

কেন এখনই বেশি ডেটা পরিষ্কার করা হচ্ছে না

চাইলেই আমরা এখন:

- কিছু অপ্রয়োজনীয় কলাম মুছে ফেলতে পারতাম
- কিছু কলামের নাম পরিবর্তন করতে পারতাম

কিন্তু সাধারণভাবে এই পর্যায়ে dataset-কে যতটা সম্ভব **raw** (অপরিবর্তিত) রাখা ভালো। কারণ পরে একই dataset বিভিন্ন ধরনের application-এ ব্যবহার করা যেতে পারে।

dataset-এ একটি গুরুত্বপূর্ণ জিনিস এখনো নেই

বর্তমান dataset-এ একটি গুরুত্বপূর্ণ তথ্য অনুপস্থিত:

Issue এবং Pull Request-এর Comments

GitHub-এ প্রতিটি issue-তে অনেক সময়:

- আলোচনা
- প্রশ্ন-উত্তর
- সমাধানের ব্যাখ্যা

এই সব comment-এ থাকে। কিন্তু সেগুলো এখনও dataset-এ যোগ করা হয়নি।

5.5.4 Dataset সমৃদ্ধ করা (Augmenting the Dataset)

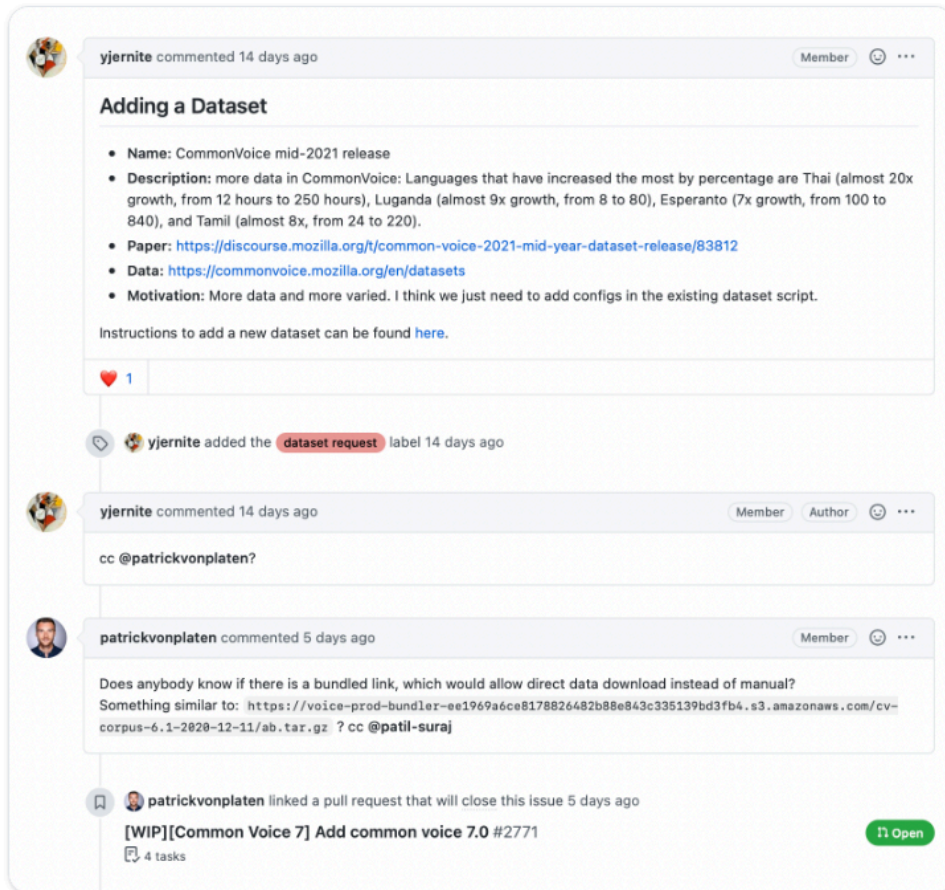
আগের ধাপে আমরা GitHub issues থেকে একটি dataset তৈরি করেছি। কিন্তু সেখানে একটি গুরুত্বপূর্ণ তথ্য এখনও যোগ করা হয়নি—**comments**।

GitHub-এ প্রতিটি **issue** বা **pull request** এর নিচে সাধারণত অনেক মন্তব্য থাকে। এই মন্তব্যগুলোতে থাকে:

- সমস্যার বিস্তারিত আলোচনা

- অন্য ডেভেলপারদের পরামর্শ
- সমাধানের ধাপ
- কোডের উদাহরণ

যদি আমরা **search engine** বা **question-answering system** তৈরি করতে চাই, তাহলে এই comments খুবই মূল্যবান তথ্যের উৎস।



GitHub API দিয়ে Comments নেওয়া

GitHub REST API-তে একটি বিশেষ endpoint আছে:

Comments endpoint

এটি ব্যবহার করে নির্দিষ্ট কোনো issue-এর সব comment পাওয়া যায়।

উদাহরণ হিসেবে ধরা হয়েছে **issue number = 2792**

issue_number = 2792

```
url = f"https://api.github.com/repos/huggingface/datasets/issues/{issue_number}/comments"

response = requests.get(url, headers=headers)
response.json()
```

এই request করলে API একটি **JSON list** রিটার্ন করে যেখানে প্রতিটি element একটি comment।

Comment ডেটার গঠন

API থেকে পাওয়া প্রতিটি comment-এ অনেক তথ্য থাকে, যেমন:

Field	অর্থ
url	API link
html_url	GitHub comment link
user	কে comment করেছে
created_at	comment কখন লেখা হয়েছে
body	comment এর আসল লেখা

সবচেয়ে গুরুত্বপূর্ণ অংশ হলো:

body field

এখানেই comment এর মূল text থাকে।

উদাহরণ:

```
@albertvillanova my tests are failing here...
AssertionError: False is not true
Any suggestions on how can I avoid this error?
```

অর্থাৎ এখানে একজন developer একটি error সম্পর্কে প্রশ্ন করছে।

Comment নেওয়ার জন্য Function লেখা

এখন একটি ছোট function লেখা হয়েছে যা একটি issue-এর সব comment নিয়ে আসবে।

```
def get_comments(issue_number):  
    url = f"https://api.github.com/repos/huggingface/datasets/issues/{issue_number}/comments"  
    response = requests.get(url, headers=headers)  
    return [r["body"] for r in response.json()]
```

এই **function** কী করে

- ১) নির্দিষ্ট issue এর comments endpoint-এ request পাঠায়
- ২) API থেকে JSON response নেয়
- ৩) প্রতিটি comment থেকে **body text** বের করে
- ৪) সব comment একটি list আকারে return করে

Function পরীক্ষা করা

```
get_comments(2792)
```

Output হবে:

```
["@albertvillanova my tests are failing here...  
AssertionError: False is not true ..."]
```

অর্থাৎ সেই issue-এর সব comment text পাওয়া যায়।

Dataset-এ Comments যোগ করা

এখন dataset-এর প্রতিটি issue-এর জন্য comments যোগ করা হবে।

```
issues_with_comments_dataset = issues_dataset.map(  
    lambda x: {"comments": get_comments(x["number"])}  
)
```

এখানে কী হচ্ছে

Dataset-এর প্রতিটি row এর জন্য:

- `x["number"]` → issue number
- `get_comments()` → সেই issue-এর comments সংগ্রহ
- নতুন column তৈরি → **comments**

ফলে dataset এখন আরও সমৃদ্ধ হয়ে যায়।

নতুন Dataset-এর কাঠামো

এখন dataset-এ থাকবে:

column	অর্থ
title	issue title
body	issue description
labels	issue label
state	open / closed
number	issue number
is_pull_request	pull request কিনা
comments	issue-এর সব comments

কেন Comments গুরুত্বপূর্ণ

Comments যোগ করলে dataset আরও শক্তিশালী হয় কারণ:

- issue discussion পাওয়া যায়
- error explanation পাওয়া যায়
- user problem বোঝা যায়
- semantic search আরও ভালো কাজ করে

উদাহরণ:

User যদি search করে:

dataset loading error

তাহলে শুধু issue title না, **comments** থেকেও **matching result** পাওয়া যাবে।

5.5.5 Dataset-টি Hugging Face Hub-এ আপলোড করা

এখন আমাদের সমৃদ্ধ **dataset** প্রস্তুত। পরের ধাপ হলো এটিকে **Hugging Face Hub**-এ আপলোড করা, যাতে অন্যরাও এটি ব্যবহার করতে পারে।

ভালো বিষয় হলো, dataset upload করা খুবই সহজ। যেমন **Transformers**-এ model বা tokenizer upload করার জন্য **push_to_hub()** ব্যবহার করা হয়, dataset-এর ক্ষেত্রেও একইভাবে **push_to_hub()** method ব্যবহার করা যায়।

প্রথমে Login করতে হবে

Dataset Hub-এ upload করার আগে তোমাকে **Hugging Face account** দিয়ে login করতে হবে। Notebook-এর ভিতরে login করার জন্য নিচের কোড ব্যবহার করা হয়:

```
from huggingface_hub import notebook_login
```

```
notebook_login()
```

এটি কী করবে?

এই command চালালে একটি **login widget** আসবে, যেখানে তুমি তোমার:

- username
- password / token

দিতে পারবে।

এরপর authentication token save হয়ে যাবে এই path-এ:

```
~/.huggingface/token
```

অর্থাৎ, একবার login করলে বারবার token লিখতে হবে না।

যদি Terminal ব্যবহার করো

যদি তুমি notebook না চালিয়ে terminal-এ কাজ করো, তাহলে CLI দিয়ে login করতে পারো:

```
huggingface-cli login
```

এখানেও token দিয়ে login করা যায়।

Dataset আপলোড করা

Login সম্পন্ন হলে খুব সহজেই dataset upload করা যাবে:

```
issues_with_comments_dataset.push_to_hub("github-issues")
```

এই লাইনের মানে কী?

এখানে `issues_with_comments_dataset` হলো তোমার তৈরি dataset,
আর `"github-issues"` হলো Hub-এ dataset repository-এর নাম।

অর্থাৎ এই command dataset-টিকে Hugging Face Hub-এ পাঠিয়ে দেবে।

অন্যরা কীভাবে **Dataset** ব্যবহার করবে

Dataset upload হয়ে গেলে, অন্য যে কেউ খুব সহজে এটি download করতে পারবে `load_dataset()` দিয়ে।

```
remote_dataset = load_dataset("lewtun/github-issues", split="train")  
remote_dataset
```

এখানে কী হচ্ছে?

- `"lewtun/github-issues"` হলো dataset repository ID
- `split="train"` মানে dataset-এর train split load করা হচ্ছে

এরপর output হিসেবে dataset-এর structure দেখা যাবে।

উদাহরণ:

```
Dataset(  
  features: [...],  
  num_rows: 2855  
)
```

অর্থাৎ dataset-এ **2855**টি row আছে।

5.5.6 Dataset Card তৈরি করা

ভালোভাবে ডকুমেন্ট করা **dataset** অন্যদের জন্য অনেক বেশি উপকারী হয়—এমনকি ভবিষ্যতে তোমার নিজের জন্যও। কারণ dataset card দেখে একজন user বুঝতে পারে:

- dataset টি কী বিষয়ে
- এটি তার কাজের জন্য উপযুক্ত কি না
- dataset এ কোনো bias আছে কি না
- dataset ব্যবহার করলে কোনো ঝুঁকি থাকতে পারে কি না

অর্থাৎ, শুধু dataset upload করলেই হয় না, সেটার পরিচয়, উৎস, ব্যবহার, সীমাবদ্ধতা—এসবও পরিষ্কারভাবে লিখে দিতে হয়।

Hugging Face Hub-এ এই তথ্য কোথায় থাকে?

Hugging Face Hub-এ dataset সম্পর্কিত এই তথ্য রাখা হয় প্রতিটি dataset repository-র:

README.md ফাইলে

অর্থাৎ dataset card মূলত একটি **README file**, যেখানে dataset সম্পর্কে বিস্তারিত লেখা থাকে।

Dataset Card তৈরির আগে কী কী করতে হবে

এই ফাইল তৈরি করার আগে সাধারণত দুটি গুরুত্বপূর্ণ ধাপ অনুসরণ করা উচিত।

১) Metadata tag তৈরি করা

প্রথম কাজ হলো **metadata tags** তৈরি করা।

এগুলো সাধারণত **YAML format**-এ লেখা হয়।

এই tags Hugging Face Hub-এ বিভিন্ন search feature-এ ব্যবহার হয়।

এর ফলে community-র অন্য সদস্যরা সহজে তোমার dataset খুঁজে পায়।

কেন এটা দরকার?

ধরো কেউ খুঁজছে:

- NLP dataset
- issue tracking dataset
- English text dataset
- classification dataset

তখন metadata tags থাকলে তোমার dataset সহজে search result-এ আসবে।

এখানে কী করতে হবে?

এজন্য ব্যবহার করতে হবে:

datasets-tagging application

যেহেতু এখানে একটি **custom dataset** তৈরি করা হয়েছে, তাই **datasets-tagging** repository clone করে application-টি লোকালি চালাতে হবে।

অর্থাৎ, tool টি ব্যবহার করে dataset-এর জন্য সঠিক tag তৈরি করতে হবে।

২) Informative Dataset Card লেখার গাইড অনুসরণ করা

দ্বিতীয় কাজ হলো:

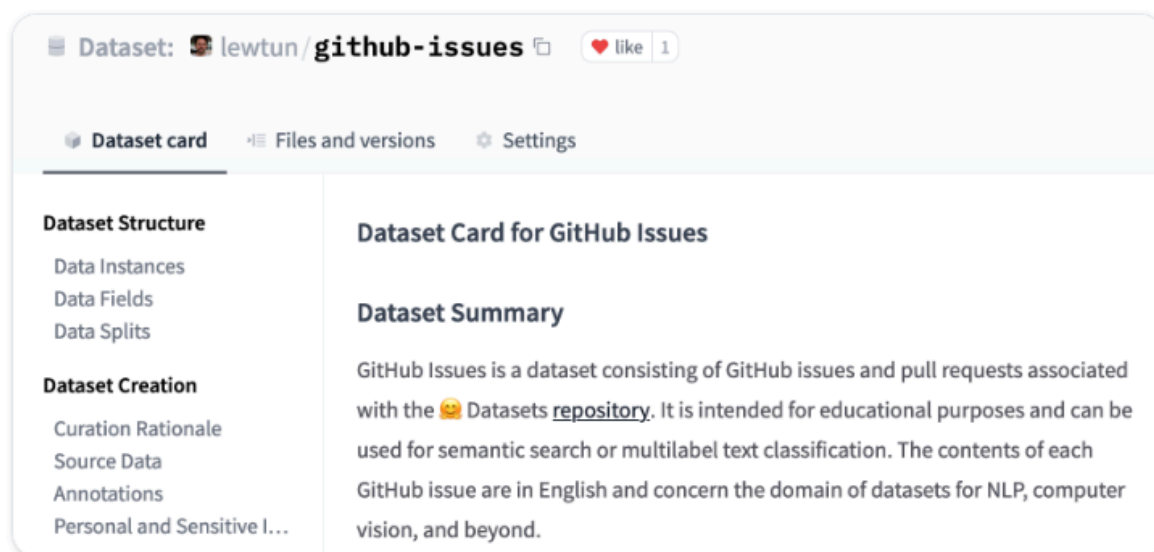
Datasets-এর guide পড়া

এই guide-এ দেখানো আছে কীভাবে একটি ভালো **dataset card** লিখতে হয়।

এটিকে template হিসেবে ব্যবহার করে তুমি README.md পূরণ করতে পারবে।

সাধারণত dataset card-এ নিচের ধরনের তথ্য থাকে:

- dataset কী
- কোথা থেকে data এসেছে
- কীভাবে dataset তৈরি করা হয়েছে
- dataset-এর structure কেমন
- কোন কাজে ব্যবহার করা যাবে
- কী limitations আছে
- bias বা risk থাকতে পারে কি না



README.md কোথায় লিখবে?

তুমি চাইলে সরাসরি **Hugging Face Hub**-এর ভেতরেই **README.md** file তৈরি করতে পারো।

এছাড়া একটি example dataset card-ও দেওয়া আছে:

lewtun/github-issues

এই repository-তে একটি template dataset card পাওয়া যাবে, যেটা দেখে নিজের dataset card বানানো সহজ হবে।

অনুশীলন

তোমাকে বলা হচ্ছে:

dataset-tagging application এবং Datasets guide ব্যবহার করে তোমার GitHub issues dataset-এর জন্য **README.md** সম্পূর্ণ করো।

অর্থাৎ এখন তোমার কাজ হবে:

- dataset-এর metadata tag তৈরি করা
- dataset card লিখা
- README.md পূরণ করা

